DOCUMENTACIÓN TÉCNICA: DASHBOARD DE INTELIGENCIA MUSICAL (SPOTIFY DATASET)

Este manual describe los pasos necesarios para configurar el entorno y ejecutar la aplicación de análisis de datos de Spotify.

1.1 Requisitos Previos

Antes de empezar, asegúrate de tener instalado:

- Python 3.9 o superior: Puedes verificarlo abriendo una terminal y escribiendo python3

```
javigonlou@pop-os:~$ python3
Python 3.12.3 (main, Mar  3 2026, 12:15:18) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

- Pip: El gestor de paquetes de Python.

1.2 Preparación del Entorno

Es fundamental aislar las librerías del proyecto para evitar conflictos con otras instalaciones.

*nota: estas capturas están sacadas en linux pero los comandos son casi parecidos

1 Crear Entorno Virtual

Comando: python -m venv .venv Crea una carpeta aislada llamada .venv que contendrá todas las librerías necesarias para el proyecto.

```
javigonlou@pop-os:~/Desktop/MEGA/programas/Python/Spotify$ python3 -m venv env
```

2 Activar el Entorno

Windows: .venv\Scripts\activate Indica a tu terminal que use el Python del entorno virtual. Verás que aparece (.venv) al principio de la línea de comandos.

```
javigonlou@pop-os:~/Desktop/MEGA/programas/Python/Spotify$ source env/bin/activate
(env) javigonlou@pop-os:~/Desktop/MEGA/programas/Python/Spotify$
```

3 Instalar Dependencias

Comando: pip install -r requirements.txt Descarga e instala automáticamente streamlit, pandas y plotly dentro de tu entorno virtual.

```
(env) javigonlou@pop-os:~/Desktop/MEGA/programas/Python/Spotify$ pip install -r requirements.txt
Collecting streamlit (from -r requirements.txt (line 1))
  Using cached streamlit-1.56.0-py3-none-any.whl.metadata (9.8 kB)
Collecting pandas (from -r requirements.txt (line 2))
  Using cached pandas-3.0.2-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (79 kB)
Collecting plotly (from -r requirements.txt (line 3))
  Using cached plotly-6.6.0-py3-none-any.whl.metadata (8.5 kB)
Collecting altair!=5.4.0,!=5.4.1,<7,>=4.0 (from streamlit->-r requirements.txt (line 1))
  Using cached altair-6.0.0-py3-none-any.whl.metadata (11 kB)
Collecting blinker<2,>=1.5.0 (from streamlit->-r requirements.txt (line 1))
  Using cached blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting cachetools<8,>=5.5 (from streamlit->-r requirements.txt (line 1))
  Using cached cachetools-7.0.5-py3-none-any.whl.metadata (5.6 kB)
Collecting click<9,>=7.0 (from streamlit->-r requirements.txt (line 1))
  Using cached click-8.3.2-py3-none-any.whl.metadata (2.6 kB)
Collecting gitpython!=3.1.19,<4,>=3.0.7 (from streamlit->-r requirements.txt (line 1))
  Using cached gitpython-3.1.46-py3-none-any.whl.metadata (13 kB)
Collecting numpy<3,>=1.23 (from streamlit->-r requirements.txt (line 1))
  Using cached numpy-2.4.4-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (6.6 kB)
Collecting packaging>=20 (from streamlit->-r requirements.txt (line 1))
  Using cached packaging-26.0-py3-none-any.whl.metadata (3.3 kB)
Collecting pillow<13,>=7.1.0 (from streamlit->-r requirements.txt (line 1))
  Using cached pillow-12.2.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (8.8 kB)
Collecting pydeck<1,>=0.8.0b4 (from streamlit->-r requirements.txt (line 1))
```

*En esta captura solo se ve una parte de las libreria de streamlit

1.3 Ejecución de la Aplicación

Hay 2 formas de iniciar el dashboard:

Opción A: Inicio Automático (Recomendado)

Ejecuta el script lanzador que gestiona la configuración por ti:

python run_app.py

Opción B: Comando Manual de Streamlit

Si prefieres lanzarlo directamente desde la terminal

streamlit run HugginfaceSpotify.py

```
(env) javigonlou@pop-os:~/Desktop/MEGA/programas/Python/Spotify$ streamlit run HugginfaceSpotify.py

  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: [redacted]
```

1.4 Estructura de Archivos del Proyecto

Para que el manual sea completo, identifica qué hace cada archivo en tu repositorio:

ARCHIVO	FUNCIÓN
HUGGINFACESPOTIFY.PY	Código principal del Dashboard. Contiene la lógica de carga, KPIs y visualizaciones.
RUN_APP.PY	Script de conveniencia que lanza el servidor de Streamlit automáticamente.
REQUIREMENTS.TXT	Listado de librerías necesarias para que el código funcione.
.VENV/ (CARPETA)	El entorno virtual con las librerías instaladas.

2.1 Identificación y Obtención de Datos

Debido a las restricciones de la API de Spotify impuestas en febrero de 2026, el proyecto utiliza una fuente de datos digital alternativa: el Hugging Face Spotify Tracks Dataset.

- Fuente: Dataset de Maharshi Pandya alojado en Hugging Face.
- Método: Obtención mediante carga directa de archivo CSV remoto a través de la librería pandas.
- Alcance: Información técnica y de popularidad de más de 114,000 canciones.

2.2 El Ciclo de Vida del Dato

La aplicación implementa el ciclo completo de gestión del dato para garantizar que la información final sea fiable:

- Obtención: Descarga automática del dataset mediante load_data().
- Procesamiento:
    - Eliminación de columnas redundantes (como Unnamed: 0).

- Segmentación de datos continuos (binning) para convertir variables como danceability en categorías legibles (dance_range).
- Agregación de datos (agrupamiento por género y rangos) para calcular promedios de popularidad.

- Análisis: Aplicación de métricas estadísticas para correlacionar atributos técnicos (energía, valencia) con el éxito comercial (popularidad).
- Visualización: Representación gráfica interactiva mediante Plotly y Streamlit para facilitar la interpretación inmediata.

3. ANÁLISIS DE KPIS Y TOMA DE DECISIONES

A continuación, se explican los 4 indicadores clave de rendimiento (KPIs) desarrollados en el dashboard:

KPI 1: Cuota de Mercado por Género (Bar Chart)

- Datos: track_genre y popularity.
- Procesamiento: Cálculo de la media de popularidad agrupada por género, filtrando los 10 mejores.
- Información extraída: Identifica qué nichos musicales tienen mayor tracción actual en la plataforma.
- Uso: Permite a una discográfica decidir en qué géneros invertir presupuesto de marketing o qué tipo de nuevos artistas contratar.

```
with col1:
    # KPI 1: Géneros Dominantes
    st.subheader("1. Cuota de Mercado por Género")
    genre_pop = data.groupby('track_genre')['popularity'].mean().sort_values(ascending=False).head(10).reset_index()
    fig1 = px.bar(genre_pop, x='popularity', y='track_genre', orientation='h', color='popularity', color_continuous_scale='Viridis')
    st.plotly_chart(fig1, width='stretch')
    st.info("Justificación: Identifica géneros para inversión publicitaria.")
```

KPI 2: Target de Audiencia y "Mood" (Scatter Plot)

- Datos: valence (positividad), energy y popularity.
- Procesamiento: Muestreo aleatorio de 1,000 canciones para visualizar la densidad sin saturar el gráfico.

- Información extraída: Clasifica las canciones en cuadrantes: Relax (baja energía), Fiesta (alta energía/valencia alta) o Melancolía (baja valencia).
- Uso: Ideal para que los “curators” de dicha plataforma diseñen playlists temáticas que mantengan una coherencia emocional para el usuario.

```
with col2:
    # KPI 2: Mood Analysis (Scatter)
    st.subheader("2. Target de Audiencia (Mood)")
    sample = data.sample(1000)
    fig2 = px.scatter(sample, x='valence', y='energy', color='popularity', hover_data=['track_name'])
    st.plotly_chart(fig2, width='stretch')
    st.info("Justificación: Define si la playlist es para Relax o Energía.")
```

KPI 3: Impacto del Contenido Explícito (Donut Chart)

- Datos: Columna booleana explicit.
- Procesamiento: Conteo de frecuencia de valores.
- Información extraída: Proporción de música apta para todos los públicos frente a contenido explícito.
- Uso: Vital para plataformas que buscan expandirse a mercados familiares o educativos, decidiendo si necesitan más contenido "Family Friendly".

```
with col3:
    # KPI 3: Contenido Explícito
    st.subheader("3. Impacto del Contenido Explícito")
    explicit_count = data['explicit'].value_counts().reset_index()
    fig3 = px.pie(explicit_count, values='count', names='explicit', hole=0.4, title="¿Es la música explícita más común?")
    st.plotly_chart(fig3, width='stretch')
    st.info("Justificación: Determina la viabilidad en canales familiares.")
```

KPI 4: Correlación Ritmo/Éxito (Line Chart)

- Datos: danceability y popularity.
- Procesamiento: Creación de 5 rangos de "bailabilidad" y cálculo de la popularidad media por rango.
- Información extraída: Determina si existe una relación directa entre qué tan bailable es una canción y su probabilidad de éxito.
- Uso: Ayuda a los productores musicales a ajustar el "tempo" y el ritmo de una pista durante la fase de composición para maximizar su potencial viral.

```
with col4:
    # KPI 4: Bailabilidad vs Popularidad
    st.subheader("4. Correlación Danceability/Éxito")
    # Agrupamos por rangos de bailabilidad
    data['dance_range'] = pd.cut(data['danceability'], bins=5)
    dance_pop = data.groupby('dance_range', observed=True)['popularity'].mean().reset_index()
    dance_pop['dance_range'] = dance_pop['dance_range'].astype(str)
    fig4 = px.line(dance_pop, x='dance_range', y='popularity', markers=True)
    st.plotly_chart(fig4, width='stretch')
    st.info("Justificación: Ayuda a decidir el ritmo de los próximos lanzamientos.")
```

4. EL VALOR DE LOS DATOS EN PLATAFORMAS DIGITALES

En los servicios tecnológicos modernos, los datos no son sólo registros; son el activo principal.

1. Personalización: Los datos de audio (como los usados en este proyecto) permiten que los algoritmos de recomendación "entiendan" el gusto del usuario sin necesidad de que el usuario etiquete cada canción manualmente.

2. Eficiencia operativa: Analizar tendencias mediante KPIs evita que las empresas inviertan en contenido que no tendrá demanda.

3. Ventaja competitiva: La capacidad de convertir datos brutos en visualizaciones claras permite reaccionar a los cambios del mercado en tiempo real, fundamental en dicha industria debido a su alta volatilidad.